

Table des matières

1	Introduction	5
1.1	Contexte du projet	5
1.2	Objectif du guide	5
1.3	Prérequis techniques	5
1.3.1	Matériel requis	5
1.3.2	Logiciels requis	5
2	Architecture Technique	6
2.1	Architecture globale	6
2.2	Protocoles de communication	6
2.2.1	Communication canne - Navigation App	6
2.2.2	Communication canne - Emergency App	6
2.3	Architecture matérielle de la canne	7
3	Réalisation et Montage du Dispositif IoT	8
3.1	Introduction	8
3.2	Conception et Réalisation Mécanique	8
3.2.1	Choix de la structure principale	8
3.2.2	Conception des pièces d'assemblage	8
3.3	Assemblage Électronique	8
3.3.1	Préparation des capteurs et composants	8
3.3.2	Intégration dans la structure	9
3.4	Réalisation du Circuit Imprimé (PCB)	9
3.4.1	Schéma fonctionnel préliminaire	9
3.4.2	Conception assistée par Megatec	9
3.4.3	Réalisation du typon sous Proteus Pro 8	9
3.4.4	Gravure de la plaque	10
3.4.5	Soudure des composants	10
3.5	Intégration du Système d'Alimentation	10
3.5.1	Caractéristiques de la batterie principale	10
3.5.2	Module de charge	10
3.5.3	Chaîne d'alimentation principale	10
3.5.4	Chaîne d'alimentation du module GSM	11
3.6	Intégration Finale	11
3.6.1	Préparation du support	11
3.6.2	Organisation du câblage	11
3.7	Fermeture et Finalisation	11
3.7.1	Tests de validation matérielle	12
3.8	Synthèse du montage	12
4	Déploiement du Firmware Embarqué	13
4.1	Configuration de l'environnement de développement	13
4.1.1	Installation Arduino IDE	13
4.1.2	Installation des drivers USB	13
4.2	Structure du projet firmware	13
4.3	Configuration du fichier Config.h	13

4.3.1	Paramètres critiques à vérifier	14
4.4	Compilation et téléversement	14
4.4.1	Configuration dans Arduino IDE	14
4.4.2	Processus de téléversement	15
4.5	Vérification du firmware	15
4.5.1	Moniteur série	15
5	Déploiement de l'application de navigation (OPEN EYES Navigation)	16
5.1	Présentation de l'application	16
5.2	Architecture technique	16
5.3	Prérequis et Installation	16
5.3.1	Environnement de développement	16
5.3.2	Vérification de l'installation	16
5.4	Configuration du Projet	17
5.4.1	Cloner et installer les dépendances	17
5.4.2	Configuration des permissions	17
5.4.3	Configuration des identifiants	18
5.5	Test en Mode Développement	19
5.5.1	Test sur émulateur/simulateur	19
5.5.2	Test en mode release	19
5.6	Génération de l'APK Android	19
5.6.1	Configuration de la signature	19
5.6.2	Générer l'APK	21
5.6.3	Tester l'APK	21
5.7	Génération de l'App Bundle (Play Store)	21
5.8	Publication sur Google Play Store	22
5.8.1	Préparation	22
5.8.2	Processus de publication	22
5.9	Génération de l'Application iOS	23
5.9.1	Prérequis (macOS uniquement)	23
5.9.2	Configuration Xcode	23
5.9.3	Build iOS	24
5.9.4	Publication sur App Store	24
5.10	Mise à Jour de l'Application	24
5.10.1	Incrémenter la version	24
5.10.2	Regénérer et republier	25
5.11	Conseils et Bonnes Pratiques	25
5.11.1	Avant de publier	25
5.11.2	Gestion des clés	26
5.11.3	Optimisation de la taille	26
5.11.4	Tests sur différents appareils	26
5.12	Dépannage	26
5.12.1	Build échoue	26
5.12.2	Permissions refusées	27
5.12.3	BLE ne fonctionne pas	27
5.13	Synthèse de déploiement d'OPEN EYES Navigation	27

6	Déploiement de l'application de gestion des urgences (OPEN EYES Emergency)	28
6.1	Présentation de l'application	28
6.2	Architecture technique	28
6.3	Déploiement du Backend (Serveur)	28
6.3.1	Prérequis serveur	28
6.3.2	Installation rapide (Développement Local)	28
6.3.3	Configuration du Module GSM	30
6.3.4	Déploiement Production (Simplifié)	31
6.4	Déploiement du Frontend (Application Mobile)	31
6.4.1	Prérequis	31
6.4.2	Configuration du projet	32
6.4.3	Génération APK Android	32
6.4.4	Génération iOS	34
6.4.5	Publication sur les stores	34
6.5	API et Points d'Intégration	34
6.5.1	Documentation API	34
6.5.2	Endpoints principaux	35
6.6	Tests et Validation	35
6.6.1	Vérifier le backend	35
6.6.2	Vérifier le module GSM	35
6.6.3	Tester l'application mobile	35
6.7	Maintenance Simplifiée	36
6.7.1	Sauvegardes PostgreSQL	36
6.7.2	Mise à jour du backend	36
6.7.3	Mise à jour de l'application mobile	36
6.8	Dépannage Rapide	36
6.8.1	Backend ne démarre pas	36
6.8.2	Module GSM ne répond pas	37
6.8.3	Application mobile ne se connecte pas	37
6.9	Checklist Déploiement	37
6.10	Synthèse OPEN EYES Emergency	38
6.11	API REST et Documentation	38
6.11.1	Accès à la documentation	38
6.11.2	Endpoints principaux	38
6.12	Checklist de déploiement en production	39
6.13	Synthèse du déploiement Emergency App	39
7	Tests et Validation	39
7.1	Procédure de tests système	39
7.1.1	Test 1 : Communication BLE	40
7.1.2	Test 2 : Détection d'obstacles	40
7.1.3	Test 3 : Détection d'eau	40
7.1.4	Test 4 : Bouton SOS	40
7.2	Résolution des problèmes courants	41

8	Maintenance et Mise à Jour	41
8.1	Mise à jour du firmware	41
8.2	Mise à jour des applications mobiles	41
8.2.1	Méthode OTA (Over-The-Air)	41
8.2.2	Méthode manuelle	41
8.3	Sauvegarde des configurations	42
9	Annexes	42
9.1	Annexe A : Commandes AT SIM808	42
9.2	Annexe B : Glossaire	42
9.3	Annexe C : Contacts support	42
10	Conclusion	43

1 Introduction

1.1 Contexte du projet

Le projet **OPEN EYES** est une solution IoT complète d'assistance pour personnes malvoyantes, composée de trois éléments principaux :

- **La canne intelligente** : dispositif physique équipé de capteurs (ultrasoniques, eau, IMU), d'un module GPS/GSM (SIM808) et d'un microcontrôleur ESP32
- **Navigation App** : application mobile Android de navigation et contrôle en temps réel
- **Emergency App** : application de gestion des contacts d'urgence et réception d'alertes

1.2 Objectif du guide

Ce guide de déploiement s'adresse aux ingénieurs et techniciens chargés de :

1. Comprendre l'architecture technique complète de la solution
2. Configurer et déployer les applications mobiles (Navigation App et Emergency App)
3. Installer le firmware embarqué sur la canne intelligente
4. Assurer la maintenance et le dépannage du système

1.3 Prérequis techniques

1.3.1 Matériel requis

- Canne intelligente OPEN EYES (assemblée et testée)
- Smartphones Android (version 9.0 minimum, Android API 28+)
- Câble USB pour programmation ESP32
- Ordinateur de développement (Windows/Linux/macOS)
- Carte SIM avec forfait data (pour module SIM808)

1.3.2 Logiciels requis

- **Pour le firmware** : Arduino IDE 2.x ou PlatformIO
- **Pour les applications** : Flutter SDK 3.x, Android Studio, JDK 11+
- **Outils complémentaires** : Git, driver USB ESP32

2 Architecture Technique

2.1 Architecture globale

La solution OPEN EYES repose sur une architecture distribuée :



FIGURE 1 – Architecture Globale du Système

2.2 Protocoles de communication

2.2.1 Communication canne - Navigation App

- **Protocole** : Bluetooth Low Energy (BLE)
- **Services BLE** :
 - Service GPS : UUID 4fafc201-...
 - Service IMU : UUID 4fafc202-...
 - Service Obstacles : UUID 4fafc203-...
 - Service Eau : UUID 4fafc204-...
- **Portée** : environ 10 mètres

2.2.2 Communication canne - Emergency App

- **Protocole** : UART/SMS via module GSM SIM808
- **Format des messages** :
 - Alerte SOS : "ALERTE SOS ! Position: [lat,lon, alt]"
 - Commandes admin :
 - Ajouter un contact : "ADMIN:ADD:+237XXXXXXXXX"
 - Supprimer un contact : "ADMIN:DEL:+237XXXXXXXXX"
 - Afficher la liste des contacts : "ADMIN:LIST"
 - Demande de localisation : "ADMIN:LOC"
 - Aide : "ADMIN:HELP"

2.3 Architecture matérielle de la canne

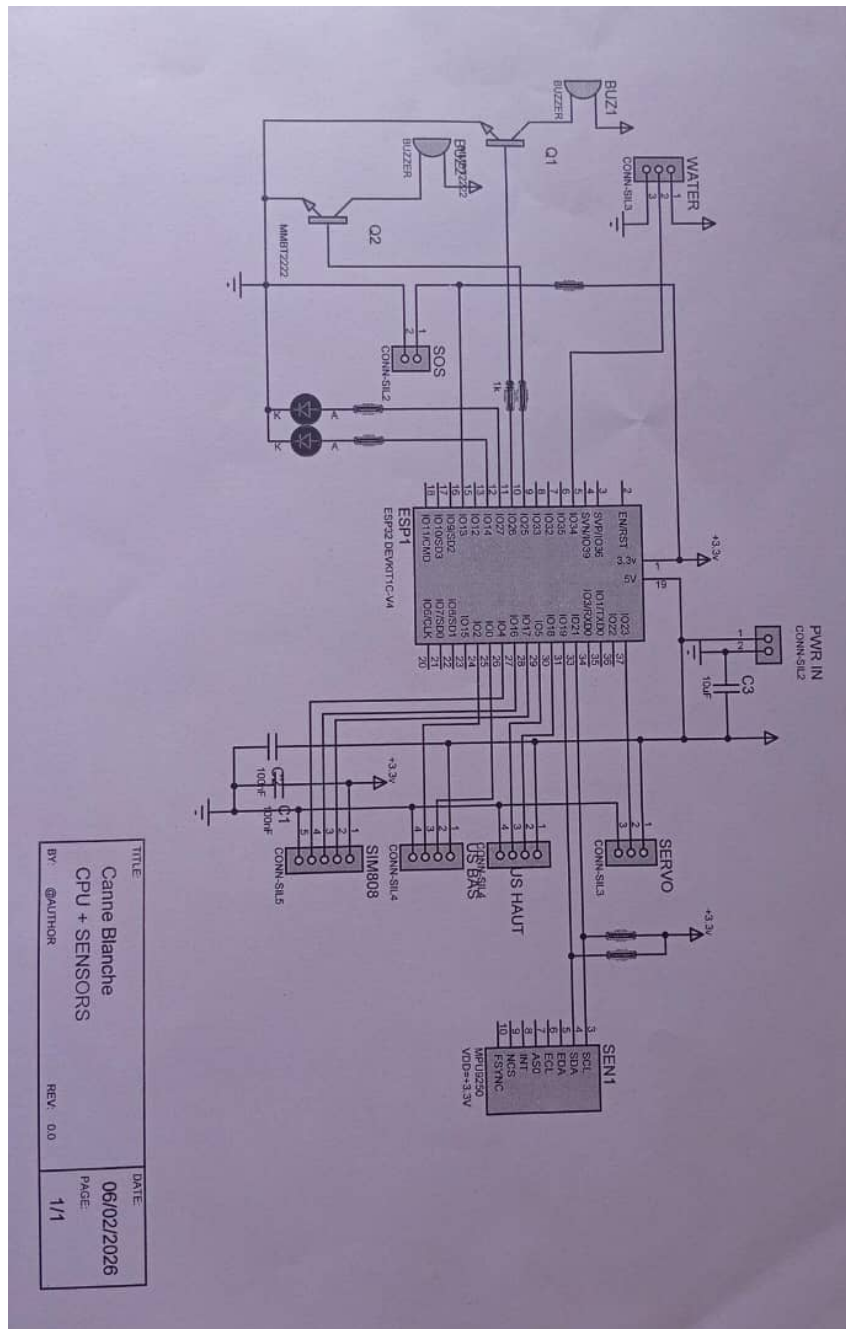


FIGURE 2 – PCB - Architecture Matérielle de la canne

3 Réalisation et Montage du Dispositif IoT

3.1 Introduction

Ce chapitre présente de manière détaillée la réalisation mécanique et électronique de la canne intelligente. L'objectif est de décrire l'ensemble des étapes depuis la conception mécanique jusqu'à l'intégration finale des composants électroniques, permettant ainsi aux techniciens et ingénieurs de comprendre la structure physique du dispositif avant de procéder au déploiement logiciel.

3.2 Conception et Réalisation Mécanique

3.2.1 Choix de la structure principale

La première étape a consisté à rechercher une tige métallique robuste servant de structure porteuse principale. Cette tige constitue l'ossature longitudinale de la canne et garantit sa rigidité mécanique.

Ensuite, un tube en PVC de diamètre adapté a été sélectionné afin de servir d'enveloppe externe et de manche ergonomique.

3.2.2 Conception des pièces d'assemblage

Des pièces mécaniques ont été modélisées puis fabriquées par impression 3D afin de :

- Fixer la tige métallique au tube PVC
- Fermer l'extrémité supérieure de la canne
- Maintenir les capteurs ultrasoniques et le cerveau moteur
- Intégrer le bouton d'alerte et l'interrupteur principal

3.3 Assemblage Électronique

3.3.1 Préparation des capteurs et composants

Chaque capteur a été préparé par soudure manuelle à l'aide d'un fer à souder.

Un code couleur normalisé a été adopté :

TABLE 1 – Code couleur des connexions

Couleur	Fonction
Noir	Masse (GND)
Rouge	Alimentation (VCC / VIN)
Bleu	Trigger (TRIG)
Vert	Echo (ECHO)

Les connexions ont été réalisées à l'aide de fils souples multibrins afin d'assurer une bonne conductivité et une flexibilité mécanique.

3.3.2 Intégration dans la structure

Des perforations ont été réalisées le long du tube PVC à l'aide d'une perceuse électrique et de la manche en PVC à l'aide d'un fer à souder afin de :

- Positionner les capteurs et le cerveau moteur
- Faire passer les câbles à l'intérieur
- Installer le bouton d'alerte et l'interrupteur

Les câbles ont ensuite été acheminés à l'intérieur de la structure vers la zone centrale d'intégration électronique.

3.4 Réalisation du Circuit Imprimé (PCB)

3.4.1 Schéma fonctionnel préliminaire

La première étape de la conception du PCB a consisté à réaliser un schéma détaillé des connexions entre les différents composants. Ce schéma préliminaire a permis d'identifier :

- Les éléments qui seraient intégrés directement sur le PCB (buzzers, LEDs, MPU9250, résistances, condensateurs)
- Les composants externes devant être connectés au PCB (ESP32, capteurs HC-SR04, moteur, capteur d'eau, module SIM808, interrupteur, bouton d'alerte)

Ce schéma fonctionnel a également défini les points de connexion pour les câbles déjà sélectionnés, en respectant le code couleur établi.

3.4.2 Conception assistée par Megatec

Une fois le schéma préliminaire finalisé, nous nous sommes rendus chez **Megatec** pour bénéficier de leur expertise. Grâce à leur accompagnement, nous avons :

- Affiné le schéma électrique final en intégrant tous les composants
- Réalisé des mesures précises de chaque composant pour dimensionner le PCB
- Défini l'emplacement des connecteurs pour les éléments hors PCB
- Validé la disposition des pistes et des composants

3.4.3 Réalisation du typon sous Proteus Pro 8

Le typon (ou *routing*) est la représentation graphique des pistes et des connexions sur le PCB. Il s'agit du plan de fabrication qui indique précisément où doivent être placés les composants et comment les pistes doivent être tracées.

Nous avons réalisé cette étape cruciale à l'aide du logiciel **Proteus Pro version 8** :

- Placement des composants selon les contraintes mécaniques et électriques
- Routage des pistes pour assurer des connexions courtes et propres
- Vérification des règles de conception (DRC - Design Rule Check)
- Génération des fichiers Gerber pour la gravure

3.4.4 Gravure de la plaque

Le typon une fois validé, nous avons procédé à la gravure de la plaque de PCB :

1. Impression du typon sur film transparent
2. Insolation de la plaque photosensible
3. Révélation chimique des pistes
4. Gravure à l'acide pour éliminer le cuivre excédentaire
5. Perçage des trous pour les composants traversants

3.4.5 Soudure des composants

Une fois la plaque gravée, nous avons procédé au soudage minutieux de tous les composants :

- Résistances et condensateurs (vérification des valeurs)
- Buzzers et LEDs
- MPU9250
- Connecteurs pour l'ESP32 (broches femelles)
- Connecteurs pour les éléments externes (borniers à vis)

Chaque soudure a été contrôlée visuellement et à l'aide d'un multimètre pour éviter les ponts de soudure.

Enfin le PCB a été validé et intégré dans la canne.

3.5 Intégration du Système d'Alimentation

3.5.1 Caractéristiques de la batterie principale

La canne est alimentée par une batterie Lithium-ion de :

- Tension nominale : 3,7 V
- Capacité : 2 Ah (2000 mAh)

3.5.2 Module de charge

La batterie a été connectée à un module de charge dédié permettant :

- La recharge sécurisée
- La protection contre les surcharges
- La protection contre les décharges profondes

3.5.3 Chaîne d'alimentation principale

L'architecture énergétique suit le schéma suivant :

1. Batterie Li-ion 3,7 V / 2 Ah
2. Module de charge intégré
3. Module élévateur DC-DC (élevateur de tension)
4. Interrupteur général à témoin lumineux

5. Distribution vers PCB principal

L'interrupteur permet d'isoler complètement le circuit afin d'économiser l'énergie lorsque la canne n'est pas utilisée.

3.5.4 Chaîne d'alimentation du module GSM

Le module GSM est également alimenté par une batterie Lithium-ion de même capacité (2000 mAh). Cette batterie est connectée à un module de charge dédié, puis directement au SIM808 pour une alimentation indépendante du reste du système.

3.6 Intégration Finale

3.6.1 Préparation du support

Une plaque en Plexiglas a été utilisée comme support mécanique pour le PCB et le module GSM.

Les câbles ont été :

- Dénudés avec précision
- Sertis à l'aide de connecteurs adaptés
- Placés dans des connecteurs
- Fixés solidement au PCB grâce aux emplacements prédéfinis

3.6.2 Organisation du câblage

TABLE 2 – Récapitulatif des connexions principales

Composant	Alimentation	Signal
ESP32 (cerveau)	3,3 V (régul. interne)	-
Capteurs HC-SR04 (x2)	5 V	GPIO
MPU9250	3,3 V	I2C
Buzzer (x2)	3,3 V	GPIO
LED (x2)	3,3 V	GPIO
Moteur	5 V	GPIO
Capteur d'eau	5 V	ADC
Module SIM808	Batterie dédiée	UART

3.7 Fermeture et Finalisation

Une fois l'ensemble des connexions validées électriquement et mécaniquement :

- Les composants ont été fixés à l'aide de vis et d'écrous, ainsi que de points de colle chaude
- Les câbles ont été organisés pour éviter les courts-circuits
- La canne a été complètement assemblée par emboîtement des sections

3.7.1 Tests de validation matérielle

Des tests fonctionnels ont été réalisés pour vérifier :

- Le bon fonctionnement des capteurs ultrasoniques (détection d'obstacles)
- L'activation du buzzer et du vibreur en cas d'alerte
- L'émission d'alerte via le module GSM
- L'autonomie énergétique conforme aux prévisions

TABLE 3 – Résultats des tests de validation matérielle

Test	Résultat attendu	Résultat obtenu
Détection obstacle < 50 cm	Alerte sonore	Validé
Détection eau	Alerte spécifique	Validé
Transmission GSM	Message reçu	Validé
Autonomie batterie	> 4 heures	Validé

3.8 Synthèse du montage

La réalisation de la canne intelligente a nécessité une intégration mécanique et électronique rigoureuse. La structure interne assure la robustesse nécessaire à une utilisation quotidienne, tandis que le système électronique garantit la détection d'obstacles et la transmission d'alertes.

La conception du PCB, depuis le schéma préliminaire jusqu'à la gravure et la soudure des composants, a constitué une étape clé du projet. L'accompagnement par Megatec a permis d'aboutir à un circuit fiable et parfaitement adapté aux contraintes mécaniques de la canne.

Le dispositif assemblé présente les caractéristiques suivantes :

- **Autonome** : batterie rechargeable intégrée
- **Robuste** : câbles protégés dans la structure
- **Ergonomique** : poids équilibré et préhension naturelle
- **Fonctionnel** : tous les capteurs répondent aux spécifications

Transition vers le déploiement logiciel

Le dispositif IoT est désormais assemblé et les tests matériels ont validé son bon fonctionnement. Les sections suivantes décrivent le déploiement du firmware embarqué sur l'ESP32 ainsi que la configuration et l'installation des applications mobiles associées.

4 Déploiement du Firmware Embarqué

4.1 Configuration de l'environnement de développement

4.1.1 Installation Arduino IDE

1. Télécharger Arduino IDE 2.x depuis <https://www.arduino.cc/en/software> ;
2. Ajouter le support ESP32 :
 - Ouvrir votre application Arduino IDE ;
 - Cliquez sur BOARDS MANAGER, représenté par la deuxième icône situé à l'extrême droite de l'application (version 2.3.7) ;
 - Rechercher puis installer la carte ESP32 Dev Module ;
3. Installer les bibliothèques requises via LIBRARY MANAGER (icône juste en-dessous de BOARDS MANAGER) :
 - ESP32Servo
 - ArduinoBLE (ou NimBLE-Arduino)
 - Adafruit MPU6050 (ou équivalent MPU9250)

4.1.2 Installation des drivers USB

- **Windows** : CP210x USB to UART Bridge VCP Drivers
- **macOS** : Installer via Homebrew ou télécharger depuis Silicon Labs
- **Linux** : Drivers généralement intégrés, ajouter l'utilisateur au groupe dialout

4.2 Structure du projet firmware

```

1 firmware/
2 |-- Main.ino           # Point d'entrée principal
3 |-- Config.h           # Configuration GPIO et paramètres
4 |-- Logger.h/cpp       # Système de logs série
5 |-- IModule.h          # Interface commune des modules
6 |-- GPSTracker.h/cpp   # Gestion GPS SIM808
7 |-- GSMEmergency.h/cpp # Gestion GSM et SMS
8 |-- ObstacleDetector.h/cpp # Capteurs ultrasoniques et eau
9 |-- GPSAssistance.h/cpp # IMU MPU9250
10 |-- BluetoothManager.h/cpp # Communication BLE
11 |-- README.md          # Documentation technique

```

Listing 1 – Arborescence du firmware

4.3 Configuration du fichier Config.h

Le fichier Config.h centralise tous les paramètres du système.

4.3.1 Paramètres critiques à vérifier

```

1 // ===== CONFIGURATION GPS/GSM (SIM808) =====
2 #define SIM808_RX 16
3 #define SIM808_TX 17
4 #define SIM808_BAUDRATE 9600
5
6 // ===== CONFIGURATION BLE =====
7 #define BLE_DEVICE_NAME "OPEN EYES"
8
9 // ===== CAPTEURS ULTRASONIQUES =====
10 #define OBSTACLE_TRIG_HIGH 19
11 #define OBSTACLE_ECHO_HIGH 18
12 #define OBSTACLE_TRIG_LOW 23
13 #define OBSTACLE_ECHO_LOW 22
14
15 // ===== CAPTEUR D'EAU =====
16 #define WATER_SENSOR_ENABLED true
17 #define WATER_SENSOR_PIN 34 // ADC1_CH6
18 #define WATER_THRESHOLD_LOW 1000
19 #define WATER_THRESHOLD_HIGH 3000
20
21 // ===== BOUTON SOS =====
22 #define BOUTON_SOS 15
23 #define DELAI_APPUI_LONG 2000 // ms
24
25 // ===== NUMERO ADMIN (a configurer) =====
26 #define ADMIN_PHONE_NUMBER "+237XXXXXXXXX"

```

Listing 2 – Extrait de Config.h

Configuration obligatoire

Avant de compiler, vous DEVEZ modifier :

- ADMIN_PHONE_NUMBER : numéro du propriétaire de la canne
- Vérifier les GPIO selon le câblage physique de votre canne
- Ajuster les seuils WATER_THRESHOLD_LOW/HIGH après tests

4.4 Compilation et téléversement

4.4.1 Configuration dans Arduino IDE

1. Sélectionner la carte : *Tools* → *Board* → *ESP32 Dev Module*
2. Configurer les paramètres :
 - Upload Speed : 921600
 - Flash Frequency : 80MHz
 - Flash Mode : QIO
 - Flash Size : 4MB (32Mb)

- Partition Scheme : Default 4MB with spiffs
 - Core Debug Level : None (ou Info pour debug)
3. Sélectionner le port série : *Tools* → *Port* → COMx (Windows) ou /dev/ttyUSBx (Linux)

4.4.2 Processus de téléversement

1. Connecter l'ESP32 via USB
2. Vérifier la connexion : *Tools* → *Get Board Info*
3. Compiler le code : *Sketch* → *Verify/Compile*
4. Téléverser : *Sketch* → *Upload*
5. Maintenir le bouton BOOT enfoncé pendant le téléversement si nécessaire

Astuce

Pour éviter le maintien manuel du bouton BOOT, installer un condensateur 10µF entre EN et GND.

4.5 Vérification du firmware

4.5.1 Moniteur série

1. Ouvrir le moniteur série : *Tools* → *Serial Monitor* ;
2. Configurer le baudrate à 9600 ;
3. Observer les logs de démarrage pour s'assurer que tout fonctionne correctement.

5 Déploiement de l'application de navigation (OPEN EYES Navigation)

5.1 Présentation de l'application

OPEN EYES Navigation est l'application mobile principale utilisée par le porteur de la canne intelligente. Elle exploite les différentes données renvoyées par les différents capteurs et le module GPS de la SIM808 via le BLE du microcontrôleur ESP32 pour assister l'utilisateur lors de sa navigation quotidienne.

5.2 Architecture technique

TABLE 4 – Stack technologique Navigation App

Composant	Description
Framework	Flutter 3.10+ (Dart) - Cross-platform
Communication	BLE (flutter_blue_plus)
Cartes	OpenStreetMap + flutter_map
GPS	geolocator (localisation native)
Voix	flutter_tts (synthèse vocale)
Permissions	permission_handler
État	Provider (gestion d'état)

Communication directe

Contrairement à l'application OPEN EYES Emergency qui passe par un backend, OPEN EYES Navigation communique **directement** avec la canne via Bluetooth BLE. Pas besoin de serveur !

5.3 Prérequis et Installation

5.3.1 Environnement de développement

1. **Flutter SDK 3.10+** : Télécharger depuis : <https://flutter.dev/docs/get-started/install>
2. **Android Studio** (pour Android) : Inclut Android SDK et émulateur
3. **Xcode 13+** (pour iOS, macOS uniquement)
4. **Git** : Pour cloner le projet

5.3.2 Vérification de l'installation

```
1 # Vérifier Flutter
2 flutter --version
3
4 # Diagnostiquer l'environnement
5 flutter doctor
```



```
6
7 # Résultat attendu (exemple):
8 # Flutter (Channel stable, 3.10.4)
9 # Android toolchain
10 # Xcode (pour iOS)
11 # Connected devices
```

Listing 3 – Vérifier Flutter

Résoudre les problèmes

Si flutter doctor affiche des erreurs :

- Installer les composants manquants (Android SDK, Xcode)
- Accepter les licences Android : `flutter doctor -android-licenses`
- Installer les outils de ligne de commande Xcode (macOS)

5.4 Configuration du Projet

5.4.1 Cloner et installer les dépendances

```
1 # Cloner le repository (exemple)
2 git clone https://github.com/votre-org/navigation_app.git
3 cd navigation_app
4
5 # Installer les dépendances Flutter
6 flutter pub get
7
8 # En cas de problème, nettoyer
9 flutter clean
10 flutter pub get
```

Listing 4 – Installation du projet

5.4.2 Configuration des permissions

Android - AndroidManifest.xml Éditer `android/app/src/main/AndroidManifest.xml` :

```
1 <manifest xmlns:android="http://schemas.android.com/apk/res/
  android">
2
3   <!-- Bluetooth Low Energy -->
4   <uses-permission android:name="android.permission.BLUETOOTH"/
    >
5   <uses-permission android:name="android.permission.
    BLUETOOTH_ADMIN"/>
6   <uses-permission android:name="android.permission.
    BLUETOOTH_SCAN"/>
7   <uses-permission android:name="android.permission.
    BLUETOOTH_CONNECT"/>
8
```

```

9      <!-- Localisation (requis pour BLE sur Android 6+) -->
10     <uses-permission android:name="android.permission.
        ACCESS_FINE_LOCATION"/>
11     <uses-permission android:name="android.permission.
        ACCESS_COARSE_LOCATION"/>
12
13     <!-- Internet (pour cartes) -->
14     <uses-permission android:name="android.permission.INTERNET"/>
15
16     <!-- Feature BLE requis -->
17     <uses-feature
18         android:name="android.hardware.bluetooth_le"
19         android:required="true"/>
20
21     <application
22         android:label="OpenEyes Navigation"
23         android:icon="@mipmap/ic_launcher">
24         <!-- ... -->
25     </application>
26 </manifest>

```

Listing 5 – Permissions Android

iOS - Info.plist Éditer ios/Runner/Info.plist :

```

1 <key>NSBluetoothAlwaysUsageDescription</key>
2 <string>Connexion Bluetooth avec la canne intelligente</string>
3
4 <key>NSLocationWhenInUseUsageDescription</key>
5 <string>Localisation pour la navigation</string>
6
7 <key>NSSpeechRecognitionUsageDescription</key>
8 <string>Commandes vocales (optionnel)</string>
9
10 <key>NSMicrophoneUsageDescription</key>
11 <string>Commandes vocales (optionnel)</string>

```

Listing 6 – Permissions iOS

5.4.3 Configuration des identifiants

Android Éditer android/app/build.gradle :

```

1 android {
2     compileSdkVersion 33
3
4     defaultConfig {
5         applicationId "com.openeyes.navigation"
6         minSdkVersion 21 // Android 5.0+ requis pour BLE
7         targetSdkVersion 33
8         versionCode 1
9         versionName "1.0.0"

```

```
10     }  
11 }
```

Listing 7 – Configuration Android

iOS Dans Xcode (open ios/Runner.xcworkspace) :

- Bundle Identifier : com.openeyes.navigation
- Version : 1.0.0
- Build : 1

5.5 Test en Mode Développement

5.5.1 Test sur émulateur/simulateur

```
1 # Lister les appareils disponibles  
2 flutter devices  
3  
4 # Lancer sur un appareil spécifique  
5 flutter run -d <device-id>  
6  
7 # Ou simplement (choisit automatiquement)  
8 flutter run
```

Listing 8 – Lancer l'application

Test BLE sur appareil réel

Le Bluetooth ne fonctionne PAS sur émulateurs! Utilisez un téléphone physique pour tester la connexion avec la canne.

5.5.2 Test en mode release

```
1 # Lancer en mode release (performance réelle)  
2 flutter run --release
```

Listing 9 – Test release

5.6 Génération de l'APK Android

5.6.1 Configuration de la signature

Créer un keystore (première fois uniquement)

```
1 keytool -genkey -v \  
2     -keystore ~/openeyes-navigation.jks \  
3     -alias openeyes-key \  
4     -keyalg RSA \  
5     -keysize 2048 \  
6     -validity 10000
```

```

7
8 # Vous serez invité à entrer :
9 # - Mot de passe du keystore
10 # - Mot de passe de la clé
11 # - Informations (nom, organisation, pays)

```

Listing 10 – Génération du keystore

Sauvegarde critique

SAUVEGARDEZ PRÉCIEUSEMENT :

- Le fichier openeyes-navigation.jks
- Le mot de passe du keystore
- Le mot de passe de la clé

Sans ces éléments, vous ne pourrez JAMAIS mettre à jour l'application sur le Play Store !

Configurer la signature dans le projet Créer android/key.properties :

```

1 storePassword=votre-mot-de-passe-keystore
2 keyPassword=votre-mot-de-passe-cle
3 keyAlias=openeyes-key
4 storeFile=/chemin/absolu/vers/openeyes-navigation.jks

```

Listing 11 – Fichier key.properties

Éditer android/app/build.gradle :

```

1 // Avant android {
2 def keystoreProperties = new Properties()
3 def keystorePropertiesFile = rootProject.file('key.properties')
4 if (keystorePropertiesFile.exists()) {
5     keystoreProperties.load(new FileInputStream(
6         keystorePropertiesFile))
7 }
8 android {
9     // ...
10
11     signingConfigs {
12         release {
13             keyAlias keystoreProperties['keyAlias']
14             keyPassword keystoreProperties['keyPassword']
15             storeFile keystoreProperties['storeFile'] ?
16                 file(keystoreProperties['storeFile']) : null
17             storePassword keystoreProperties['storePassword']
18         }
19     }
20
21     buildTypes {
22         release {

```

```
23         signingConfig signingConfigs.release
24         minifyEnabled true
25         shrinkResources true
26     }
27 }
28 }
```

Listing 12 – Configuration signature dans build.gradle

5.6.2 Générer l'APK

```
1 # Générer l'APK signé
2 flutter build apk --release
3
4 # Fichier généré:
5 # build/app/outputs/flutter-apk/app-release.apk
```

Listing 13 – Build APK

5.6.3 Tester l'APK

```
1 # Installer directement sur un appareil connecté
2 flutter install
3
4 # Ou copier manuellement l'APK
5 # Sur le téléphone: autoriser "Sources inconnues"
6 # Ouvrir le fichier app-release.apk
```

Listing 14 – Installation APK

5.7 Génération de l'App Bundle (Play Store)

APK vs AAB

- **APK** : Installation directe, fichier unique
- **AAB (App Bundle)** : Recommandé pour Play Store, Google optimise l'APK pour chaque appareil

```
1 # Générer l'App Bundle signé
2 flutter build appbundle --release
3
4 # Fichier généré:
5 # build/app/outputs/bundle/release/app-release.aab
```

Listing 15 – Build App Bundle

5.8 Publication sur Google Play Store

5.8.1 Préparation

Créer un compte développeur

1. Aller sur <https://play.google.com/console>
2. Payer les frais uniques (25 USD)
3. Vérifier votre identité

TABLE 5 – Assets requis pour Play Store

Élément	Spécifications
Icône application	512×512 px (PNG)
Feature graphic	1024×500 px (JPG ou PNG)
Captures d'écran téléphone	2-8 images, 16 :9, min 1080×1920 px
Description courte	80 caractères max
Description complète	4000 caractères max
Catégorie	Outils, Accessibilité, ou Santé
Classification contenu	E (Tout public)
Politique confidentialité	URL valide

Préparer les assets

5.8.2 Processus de publication

Créer l'application

1. Dans Play Console : *Créer une application*
2. Nom : "OpenEyes Navigation"
3. Langue par défaut : Français
4. Type : Application
5. Gratuite ou payante : Gratuite

Remplir la fiche boutique

1. Détails de l'application

- Description courte et complète
- Icône et feature graphic
- Captures d'écran
- Catégorie : Accessibilité

2. Classification du contenu

- Questionnaire sur le contenu
- Classification par âge

3. Politique de confidentialité

- URL de votre politique
- Détails sur les données collectées

4. Tarification et distribution

- Pays de distribution
- Conformité avec les lois (RGPD, etc.)

Téléverser l'App Bundle

1. Aller dans *Release* → *Production*
2. Cliquer sur *Créer une version*
3. Télécharger `app-release.aab`
4. Ajouter les notes de version (nouveau, corrections)
5. Vérifier et lancer le déploiement

Attendre la validation

- Durée : quelques heures à 3-5 jours
- Vous recevrez un email quand l'app sera :
 - Approuvée → En ligne sur Play Store
 - Rejetée → Corriger et re-soumettre

5.9 Génération de l'Application iOS

5.9.1 Prérequis (macOS uniquement)

- macOS 11+ avec Xcode 13+
- Compte Apple Developer (99 USD/an)
- Certificats de signature configurés

5.9.2 Configuration Xcode

```
1 cd ios
2 pod repo update
3 pod install
4 cd ..
```

Listing 16 – Installation pods iOS

Ouvrir dans Xcode :

```
1 open ios/Runner.xcworkspace
```

Dans Xcode :

1. Sélectionner *Runner* → *Signing & Capabilities*
2. Choisir votre *Team* (compte Apple Developer)
3. Vérifier le *Bundle Identifier* : `com.openeyes.navigation`
4. Activer *Automatically manage signing*

5.9.3 Build iOS

```
1 # Build pour dispositif physique
2 flutter build ios --release
```

Listing 17 – Build iOS release

5.9.4 Publication sur App Store

Créer l'application dans App Store Connect

1. Aller sur <https://appstoreconnect.apple.com>
2. *Mes apps* → + → *Nouvelle app*
3. Remplir : nom, langue, Bundle ID, SKU

Archive et soumission Dans Xcode :

1. *Product* → *Archive*
2. Attendre la fin de l'archivage
3. Dans Organizer : *Distribute App*
4. Choisir *App Store Connect*
5. Suivre l'assistant

Remplir la fiche App Store

- Description, captures d'écran, catégorie
- Politique de confidentialité
- Informations de contact
- Soumission pour review

5.10 Mise à Jour de l'Application

5.10.1 Incrémenter la version

Éditer pubspec.yaml :

```
1 version: 1.1.0+2
2 #       ^^^^^ ^^^
3 #       |       |
4 #       |       +--- buildNumber (versionCode Android)
5 #       +----- version (versionName Android,
        CFBundleShortVersionString iOS)
```


Semantic Versioning

Format : MAJOR.MINOR.PATCH+BUILD

- **MAJOR** : Changements incompatibles (ex : 2.0.0)
- **MINOR** : Nouvelles fonctionnalités (ex : 1.1.0)
- **PATCH** : Corrections de bugs (ex : 1.0.1)
- **BUILD** : Numéro de build (doit toujours augmenter)

5.10.2 Régénérer et republier

```
1 # Android
2 flutter build appbundle --release
3
4 # iOS
5 flutter build ios --release
6
7 # Puis suivre le même processus de publication
```

Listing 18 – Mise à jour

5.11 Conseils et Bonnes Pratiques

5.11.1 Avant de publier

- Tester en mode `-release` sur appareil physique
- Vérifier les permissions (BLE, localisation)
- Tester la connexion BLE avec la canne réelle
- Vérifier que les cartes s'affichent (connexion Internet)
- Tester la synthèse vocale
- Icône et nom de l'app corrects
- Version incrémentée dans `pubspec.yaml`
- Keystore sauvegardé en lieu sûr
- Politique de confidentialité rédigée et accessible
- Captures d'écran de qualité (sans données sensibles)

5.11.2 Gestion des clés

Sécurité des clés

NE JAMAIS :

- Committer `key.properties` ou `.jks` dans Git
- Partager les mots de passe en clair
- Perdre le keystore (impossible de mettre à jour l'app)

TOUJOURS :

- Sauvegarder le keystore sur 2+ emplacements sécurisés
- Utiliser un gestionnaire de mots de passe
- Ajouter `*.jks` et `key.properties` au `.gitignore`

5.11.3 Optimisation de la taille

```
1 # Build avec optimisations
2 flutter build apk --release --split-per-abi
3
4 # Génère 3 APK optimisés:
5 # - armeabi-v7a (32-bit ARM)
6 # - arm64-v8a (64-bit ARM)
7 # - x86_64 (émulateurs)
```

Listing 19 – Réduire la taille de l'APK

5.11.4 Tests sur différents appareils

- Tester sur Android 5.0 (minSdkVersion 21) minimum
- Tester sur plusieurs résolutions d'écran
- Vérifier les performances sur appareils anciens
- Tester les permissions sur Android 12+

5.12 Dépannage

5.12.1 Build échoue

```
1 # Nettoyer tout
2 flutter clean
3 cd android && ./gradlew clean && cd ..
4 flutter pub get
5 flutter build apk --release -v # -v pour logs détaillés
```

Listing 20 – Nettoyage complet

5.12.2 Permissions refusées

- Vérifier AndroidManifest.xml et Info.plist
- Tester sur appareil réel (pas émulateur pour BLE)
- Demander explicitement les permissions dans le code

5.12.3 BLE ne fonctionne pas

- Vérifier que BLUETOOTH_SCAN et BLUETOOTH_CONNECT sont dans le manifest
- Sur Android 12+, demander les permissions au runtime
- Activer la localisation (requis pour BLE sur Android)

5.13 Synthèse de déploiement d'OPEN EYES Navigation

La Navigation App est une application Flutter cross-platform qui communique directement avec la canne via Bluetooth BLE. Sa publication nécessite :

- Configuration des permissions BLE et localisation
- Signature sécurisée avec keystore (Android) ou certificat (iOS)
- Génération d'un APK/AAB (Android) ou Archive (iOS)
- Publication via Play Store ou App Store avec assets et description
- Tests rigoureux en mode release sur appareils réels

6 Déploiement de l'application de gestion des urgences (OPEN EYES Emergency)

6.1 Présentation de l'application

OPEN EYES Emergency est une solution composite backend + frontend destinée aux contacts d'urgence. Elle est destinée à l'assistant du malvoyant afin de permettre à se dernier d'intervenir en cas d'urgence.

6.2 Architecture technique

TABLE 6 – Composants de l'Emergency App

Composant	Description
Backend	Django 4.2 + Django REST Framework
Base de données	PostgreSQL (stockage alertes, contacts, positions)
Cache/Queue	Redis + Celery (tâches asynchrones SMS)
Module GSM	SIM800/SIM808 USB (réception/envoi SMS)
Frontend	Flutter 3.10+ (Android/iOS)
API	REST JSON + JWT (authentification sécurisée)

6.3 Déploiement du Backend (Serveur)

6.3.1 Prérequis serveur

- **Serveur Linux** : Ubuntu 20.04+ ou Debian 11+ (recommandé)
- **Python 3.10+**
- **PostgreSQL 12+**
- **Redis 6+**
- **Module GSM USB** : SIM800 ou SIM808 avec carte SIM active
- **Accès root** ou **sudo** sur le serveur

6.3.2 Installation rapide (Développement Local)

Étape 1 : Installer les dépendances système

```

1 # Mettre à jour le système
2 sudo apt update && sudo apt upgrade -y
3
4 # Installer Python, PostgreSQL, Redis
5 sudo apt install -y python3.10 python3-pip \
6     postgresql postgresql-contrib redis-server git

```

Listing 21 – Installation Ubuntu/Debian

Étape 2 : Configurer PostgreSQL

```
1 # Se connecter à PostgreSQL
2 sudo -u postgres psql
3
4 # Dans le shell PostgreSQL:
5 CREATE DATABASE smart_cane_db;
6 CREATE USER smartcane WITH PASSWORD 'votre-mot-de-passe';
7 GRANT ALL PRIVILEGES ON DATABASE smart_cane_db TO smartcane;
8 \q
```

Listing 22 – Configuration base de données

Étape 3 : Cloner et configurer le projet

```
1 # Cloner le projet
2 git clone <url-du-repo> emergency_backend
3 cd emergency_backend/backend
4
5 # Créer l'environnement virtuel Python
6 python3 -m venv venv
7 source venv/bin/activate # Linux/macOS
8 # venv\Scripts\activate # Windows
9
10 # Installer les dépendances Python
11 pip install -r requirements.txt
```

Listing 23 – Installation du backend

Étape 4 : Configurer les variables d'environnement Créer un fichier `.env` dans `backend/` :

```
1 # Django
2 SECRET_KEY=changez-moi-en-production-par-une-cle-aleatoire
3 DEBUG=True # False en production !
4 ALLOWED_HOSTS=localhost,127.0.0.1
5
6 # Base de données
7 DB_NAME=smart_cane_db
8 DB_USER=smartcane
9 DB_PASSWORD=votre-mot-de-passe
10 DB_HOST=localhost
11 DB_PORT=5432
12
13 # Redis (tâches asynchrones)
14 CELERY_BROKER_URL=redis://localhost:6379/0
15
16 # Module GSM
17 GSM_PORT=/dev/ttyACM0 # ou COM3 sous Windows
18 GSM_BAUD_RATE=9600
```

Listing 24 – Fichier `.env`

Sécurité Production

En production, remplacez :

- SECRET_KEY par une clé générée aléatoirement (50+ caractères)
- DEBUG=False
- ALLOWED_HOSTS par votre nom de domaine
- Tous les mots de passe par des valeurs sécurisées

Étape 5 : Initialiser la base de données

```
1 # Créer les tables
2 python manage.py migrate
3
4 # Créer un compte admin
5 python manage.py createsuperuser
6 # Entrer: nom d'utilisateur, email, mot de passe
7
8 # Collecter les fichiers statiques (CSS, JS)
9 python manage.py collectstatic --noinput
```

Listing 25 – Migrations Django

Étape 6 : Lancer les services

```
1 # Terminal 1 - Serveur Django
2 python manage.py runserver 0.0.0.0:8000
3
4 # Terminal 2 - Celery Worker (tâches SMS)
5 celery -A config worker -l info
6
7 # Terminal 3 - Celery Beat (scheduler)
8 celery -A config beat -l info
```

Listing 26 – Démarrage développement

Le serveur sera accessible à : <http://localhost:8000>

Admin Django : <http://localhost:8000/admin>

6.3.3 Configuration du Module GSM

Identifier le port USB

```
1 # Linux
2 ls -la /dev/tty* | grep USB
3 # Résultat typique: /dev/ttyUSB0 ou /dev/ttyACM0
4
5 # Windows (dans le Gestionnaire de périphériques)
6 # Chercher "Ports (COM et LPT)"
7 # Exemple: COM3, COM4
```

Listing 27 – Trouver le module GSM

Tester la communication

```
1 # Linux - installer picocom
2 sudo apt install picocom
3
4 # Se connecter au module
5 picocom -b 9600 /dev/ttyACM0
6
7 # Taper les commandes AT:
8 AT
9 # Devrait répondre: OK
10
11 AT+CSQ
12 # Devrait retourner: +CSQ: 15,0 (qualité signal)
13
14 # Quitter: Ctrl+A puis Ctrl+X
```

Listing 28 – Test module GSM

Donner les permissions

```
1 # Linux - ajouter votre utilisateur au groupe dialout
2 sudo usermod -a -G dialout $USER
3
4 # Redémarrer pour appliquer
5 # Ou: newgrp dialout
```

Listing 29 – Permissions port série

6.3.4 Déploiement Production (Simplifié)

Pour un déploiement production complet, vous aurez besoin de :

1. **Serveur web** : Nginx ou Apache comme reverse proxy
2. **WSGI server** : Gunicorn pour servir Django
3. **Process manager** : Supervisor pour gérer les services
4. **SSL/HTTPS** : Certificat Let's Encrypt
5. **Sauvegardes** : PostgreSQL backup automatisé

Documentation détaillée

Le déploiement production complet (Nginx, Gunicorn, Supervisor, SSL) est détaillé dans la documentation technique complète du projet. Pour un usage initial, le mode développement suffit.

6.4 Déploiement du Frontend (Application Mobile)

6.4.1 Prérequis

- Flutter SDK 3.10+ (voir section Navigation App)
- Android Studio (pour Android)
- Xcode 13+ (pour iOS, macOS uniquement)

6.4.2 Configuration du projet

Installer les dépendances

```
1 cd emergency_backend/frontend
2
3 # Installer les packages Flutter
4 flutter pub get
5
6 # Vérifier l'environnement
7 flutter doctor
```

Listing 30 – Installation Frontend

Configurer l'URL de l'API Éditer lib/config.dart :

```
1 class ApiConfig {
2   // Développement local
3   static const String baseUrl = 'http://localhost:8000';
4
5   // Production (à changer !)
6   // static const String baseUrl = 'https://votre-domaine.com';
7
8   static const String apiPath = '/api/';
9   static const Duration timeout = Duration(seconds: 30);
10 }
```

URL Production

Avant de publier, changez `baseUrl` pour pointer vers votre serveur de production (HTTPS obligatoire)!

6.4.3 Génération APK Android

Configuration de base Éditer android/app/build.gradle :

```
1 android {
2   compileSdkVersion 33
3
4   defaultConfig {
5     applicationId "com.openeyes.emergency"
6     minSdkVersion 21
7     targetSdkVersion 33
8     versionCode 1
9     versionName "1.0.0"
10  }
11 }
```

Listing 31 – Configuration Android Emergency App

Créer le keystore


```

1 keytool -genkey -v \
2   -keystore ~/openeyes-emergency.jks \
3   -alias emergency-key \
4   -keyalg RSA \
5   -keysize 2048 \
6   -validity 10000
7
8 # Sauvegarder le fichier .jks et les mots de passe !

```

Listing 32 – Keystore Emergency App

Configurer la signature Créer android/key.properties :

```

1 storePassword=votre-mot-de-passe-keystore
2 keyPassword=votre-mot-de-passe-cle
3 keyAlias=emergency-key
4 storeFile=/chemin/absolu/vers/openeyes-emergency.jks

```

Listing 33 – key.properties

Éditer android/app/build.gradle (ajouter avant android {) :

```

1 def keystoreProperties = new Properties()
2 def keystorePropertiesFile = rootProject.file('key.properties')
3 if (keystorePropertiesFile.exists()) {
4     keystoreProperties.load(new FileInputStream(
5         keystorePropertiesFile))
6 }
7 android {
8     // ...
9
10    signingConfigs {
11        release {
12            keyAlias keystoreProperties['keyAlias']
13            keyPassword keystoreProperties['keyPassword']
14            storeFile file(keystoreProperties['storeFile'])
15            storePassword keystoreProperties['storePassword']
16        }
17    }
18
19    buildTypes {
20        release {
21            signingConfig signingConfigs.release
22        }
23    }
24 }

```

Listing 34 – Signature dans build.gradle

Générer l'APK

```

1 # APK pour distribution directe

```

```
2 flutter build apk --release
3
4 # App Bundle pour Play Store
5 flutter build appbundle --release
6
7 # Fichiers générés:
8 # APK: build/app/outputs/flutter-apk/app-release.apk
9 # AAB: build/app/outputs/bundle/release/app-release.aab
```

Listing 35 – Build Emergency APK

6.4.4 Génération iOS

```
1 # Installer les pods
2 cd ios && pod install && cd ..
3
4 # Build iOS
5 flutter build ios --release
6
7 # Ouvrir dans Xcode pour archiver
8 open ios/Runner.xcworkspace
9 # Product > Archive > Distribute App
```

Listing 36 – Build Emergency iOS

6.4.5 Publication sur les stores

Le processus de publication est identique à la Navigation App :

1. **Play Store** : Uploader l'AAB, remplir la fiche boutique
2. **App Store** : Archiver dans Xcode, soumettre via App Store Connect
3. **Distribution directe** : Partager l'APK (Android uniquement)

Référence

Voir la section *Déploiement de la Navigation App* pour les détails complets de publication (assets, descriptions, validation).

6.5 API et Points d'Intégration

6.5.1 Documentation API

Le backend Django expose une API REST documentée automatiquement :

- **Swagger UI** : <http://localhost:8000/api/schema/swagger/>
- **Admin Django** : <http://localhost:8000/admin/>

6.5.2 Endpoints principaux

TABLE 7 – API Emergency - Endpoints clés

Endpoint	Méthode	Description
/api/auth/login/	POST	Connexion utilisateur
/api/alerts/	GET	Liste des alertes SOS
/api/positions/	GET	Positions GPS historiques
/api/contacts/	GET, POST, DELETE	Gestion contacts d'urgence
/api/canes/	GET	Informations cannes enregistrées
/api/health/	GET	Santé du serveur

6.6 Tests et Validation

6.6.1 Vérifier le backend

```

1 # Vérifier que Django fonctionne
2 curl http://localhost:8000/api/health/
3
4 # Vérifier les logs
5 tail -f logs/django.log
6
7 # Tester l'admin Django (navigateur)
8 # http://localhost:8000/admin

```

Listing 37 – Tests backend

6.6.2 Vérifier le module GSM

```

1 # Tester la réception SMS
2 # 1. Envoyer un SMS au numéro de la carte SIM du serveur
3 # 2. Vérifier les logs Celery
4
5 tail -f logs/celery_worker.log
6 # Devrait afficher: "SMS reçu de +237XXXXXXXXX"

```

Listing 38 – Test GSM

6.6.3 Tester l'application mobile

1. Lancer le backend (serveur accessible)
2. Installer l'APK sur un téléphone
3. Créer un compte utilisateur
4. Vérifier la connexion à l'API
5. Simuler une alerte SOS (via admin Django ou SMS réel)
6. Vérifier que l'alerte apparaît dans l'app

6.7 Maintenance Simplifiée

6.7.1 Sauvegardes PostgreSQL

```
1 # Sauvegarde manuelle
2 pg_dump -U smartcane smart_cane_db > \
3     backup_$(date +%Y%m%d).sql
4
5 # Restauration
6 psql -U smartcane smart_cane_db < backup_20260302.sql
```

Listing 39 – Backup base de données

6.7.2 Mise à jour du backend

```
1 cd emergency_backend
2
3 # Pull les derniers changements
4 git pull origin main
5
6 # Activer l'environnement virtuel
7 source backend/venv/bin/activate
8
9 # Installer nouvelles dépendances
10 pip install -r backend/requirements.txt
11
12 # Appliquer migrations
13 python backend/manage.py migrate
14
15 # Redémarrer les services (développement)
16 # Ctrl+C puis relancer runserver, celery worker, celery beat
```

Listing 40 – Mise à jour serveur

6.7.3 Mise à jour de l'application mobile

1. Incrémenter la version dans `pubspec.yaml` : `1.1.0+2`
2. Rebuild : `flutter build apk -release`
3. Redistribuer l'APK ou publier la mise à jour sur les stores

6.8 Dépannage Rapide

6.8.1 Backend ne démarre pas

```
1 # Vérifier PostgreSQL
2 sudo systemctl status postgresql
3
4 # Vérifier Redis
5 redis-cli ping
6 # Devrait répondre: PONG
```

```
7
8 # Vérifier les migrations
9 python manage.py showmigrations
```

Listing 41 – Debug backend

6.8.2 Module GSM ne répond pas

```
1 # Vérifier le port
2 ls -la /dev/ttyACM0
3
4 # Tester manuellement
5 picocom -b 9600 /dev/ttyACM0
6 # Entrer: AT
7 # Devrait répondre: OK
```

Listing 42 – Debug GSM

6.8.3 Application mobile ne se connecte pas

1. Vérifier que le backend fonctionne : `curl http://localhost:8000/api/health/`
2. Vérifier l'URL dans `lib/config.dart`
3. Si erreur CORS : ajouter l'origine dans Django settings
4. Vérifier les logs serveur : `tail -f logs/django.log`

6.9 Checklist Déploiement

Backend

- PostgreSQL installé et configuré
- Redis installé et fonctionnel
- Django settings configurés (.env)
- Migrations appliquées
- Superutilisateur créé
- Module GSM détecté et testé
- Celery worker et beat démarrés

Frontend

- URL API correcte (config.dart)
- Keystore créé et sauvegardé
- APK/AAB généré en mode release
- Application testée sur appareil réel
- Permissions accordées

6.10 Synthèse OPEN EYES Emergency

L'Emergency App est une solution backend + frontend qui centralise les alertes SOS des cannes intelligentes via SMS. Points clés :

- **Backend Django** : Serveur qui reçoit les SMS, stocke les alertes, expose une API REST
- **Module GSM** : Communication SMS bidirectionnelle (réception alertes, envoi commandes)
- **Frontend Flutter** : Application mobile qui consomme l'API pour afficher alertes et positions
- **Architecture centralisée** : Permet aux contacts d'urgence de recevoir les alertes à distance

Différences clés avec Navigation App

Caractéristique	Navigation	Emergency
Communication	BLE direct	SMS via serveur
Backend requis	Non	Oui (Django)
Portée	10 mètres	Illimitée (réseau GSM)
Utilisateur	Porteur de la canne	Contacts d'urgence
Complexité déploiement	Simple	Moyenne

6.11 API REST et Documentation

Le backend Django expose une API REST documentée automatiquement.

6.11.1 Accès à la documentation

- **Swagger UI** : <https://yourdomain.com/api/schema/swagger/>
- **ReDoc** : <https://yourdomain.com/api/schema/redoc/>
- **Admin Django** : <https://yourdomain.com/admin/>

6.11.2 Endpoints principaux

TABLE 8 – Principaux endpoints de l'API Emergency

Endpoint	Méthode	Description
/api/auth/login/	POST	Authentification utilisateur
/api/auth/register/	POST	Inscription utilisateur
/api/canes/	GET, POST	Liste et création de cannes
/api/contacts/	GET, POST, PUT, DELETE	Gestion des contacts d'urgence
/api/positions/	GET	Historique des positions GPS
/api/alerts/	GET	Historique des alertes SOS
/api/sms/send/	POST	Envoi de commandes SMS
/api/health/	GET	Vérification santé du serveur

6.12 Checklist de déploiement en production

- DEBUG = False dans Django settings
- SECRET_KEY généré et sécurisé
- HTTPS configuré avec certificat SSL/TLS valide
- CORS configuré pour domaines autorisés uniquement
- PostgreSQL sauvegardé automatiquement (cron)
- Redis monitoring actif
- Logs centralisés et rotatifs
- Module GSM testé et fonctionnel
- Tâches Celery (worker + beat) démarrées
- Supervisor configuré et services auto-restart
- Nginx configuré avec reverse proxy
- API documentation accessible
- Frontend Flutter build en release (pas debug)
- URL API correcte dans config.dart
- Permissions Android/iOS accordées
- Tests end-to-end réalisés

6.13 Synthèse du déploiement Emergency App

Le déploiement de l'Emergency App nécessite la mise en place d'une infrastructure backend complète (Django + PostgreSQL + Redis + Celery) ainsi que la compilation et distribution des applications mobiles Flutter.

Points clés à retenir :

- Le backend Django gère toute la logique métier et communique avec la canne via SMS (module GSM)
- Les applications mobiles Flutter consomment l'API REST sécurisée par JWT
- Celery assure le traitement asynchrone des SMS et des tâches planifiées
- La supervision via Supervisor et le monitoring des logs sont essentiels en production
- Les sauvegardes PostgreSQL doivent être automatisées et testées régulièrement

Prochaine étape

Une fois l'Emergency App déployée, vous pouvez procéder aux tests système complets qui valideront l'interaction entre la canne, la Navigation App, et l'Emergency App.

7 Tests et Validation

7.1 Procédure de tests système

7.1.1 Test 1 : Communication BLE

1. Allumer la canne
2. Lancer la Navigation App
3. Vérifier que "OPEN EYES" apparaît dans le scan BLE
4. Connexion réussie : icône Bluetooth passe au vert
5. Vérifier que les données GPS s'affichent (délai 30-60s pour fix initial)

Résultat attendu : Coordonnées GPS différentes de 0.000000

7.1.2 Test 2 : Détection d'obstacles

1. Placer un obstacle à moins de 30 cm devant le capteur haut
2. Observer le buzzer 1 : doit émettre des bips
3. Observer l'application : obstacle affiché en rouge
4. Éloigner l'obstacle : bips s'arrêtent

Résultat attendu : Bip continu si < 10 cm, bips espacés si 10-30 cm

7.1.3 Test 3 : Détection d'eau

1. Tremper la sonde du capteur d'eau dans de l'eau
2. Observer le buzzer 2 : doit émettre 3 bips rapides
3. Vérifier les logs série : "EAU DÉTECTÉE! Niveau=XXXX"

Résultat attendu : Valeur ADC > 1000 (ajustable selon le capteur)

7.1.4 Test 4 : Bouton SOS

1. Maintenir le bouton SOS enfoncé pendant 2 secondes
2. Observer les logs : "APPUI LONG DÉTECTÉ - ALERTE SOS"
3. Vérifier que les contacts d'urgence reçoivent le SMS

Résultat attendu : SMS contenant "ALERTE SOS! Position : [lat,lon]"

7.2 Résolution des problèmes courants

TABLE 9 – Dépannage des problèmes fréquents

Problème	Solution
ESP32 ne se programme pas	Maintenir bouton BOOT pendant upload ; vérifier drivers USB
BLE invisible	Vérifier BLE_DEVICE_NAME dans Config.h ; ESP32 redémarré ?
GPS à 0.000000	Attendre 60s dehors ; vérifier antenne SIM808 ; logs +CGPSINF ?
Buzzers ne sonnent pas	Vérifier GPIO ; tester avec digitalWrite manuel
Capteur eau ne détecte rien	Vérifier GPIO34 ; ajuster WATER_THRESHOLD_LOW
SOS ne déclenche pas SMS	Vérifier carte SIM insérée ; crédit disponible ; réseau GSM ?

8 Maintenance et Mise à Jour

8.1 Mise à jour du firmware

Pour mettre à jour le firmware de la canne :

1. Connecter la canne via USB
2. Ouvrir le nouveau code dans Arduino IDE
3. Vérifier les modifications dans Config.h (ne pas écraser les paramètres personnalisés)
4. Compiler et téléverser
5. Vérifier les logs série pour confirmer le bon fonctionnement

8.2 Mise à jour des applications mobiles

8.2.1 Méthode OTA (Over-The-Air)

Si un système de mise à jour automatique a été implémenté :

- L'application vérifie automatiquement les nouvelles versions
- L'utilisateur est notifié et peut installer la mise à jour

8.2.2 Méthode manuelle

1. Générer le nouvel APK : `flutter build apk -release`
2. Incrémenter le numéro de version dans `pubspec.yaml`
3. Transférer l'APK sur le smartphone
4. Installer par-dessus l'ancienne version (les données sont conservées)

8.3 Sauvegarde des configurations

Bonnes pratiques

Avant toute mise à jour, sauvegarder :

- Liste des contacts d'urgence (via SMS "ADMIN :LIST")
- Paramètres personnalisés dans Config.h
- Données de calibration des capteurs

9 Annexes

9.1 Annexe A : Commandes AT SIM808

TABLE 10 – Principales commandes AT du SIM808

Commande	Description
AT	Test de communication
AT+CGPSPWR=1	Activer le GPS
AT+CGPSINF=32	Lire les données GPS complètes
AT+CMGF=1	Mode texte pour SMS
AT+CMGS="+237XXX"	Envoyer un SMS
AT+CSQ	Qualité du signal GSM

9.2 Annexe B : Glossaire

APK Android Package Kit, format de fichier exécutable Android

BLE Bluetooth Low Energy, protocole Bluetooth basse consommation

GPIO General Purpose Input/Output, broches programmables

UART Universal Asynchronous Receiver-Transmitter, protocole série

I2C Inter-Integrated Circuit, bus de communication série

ADC Analog-to-Digital Converter, convertisseur analogique-numérique

PWM Pulse Width Modulation, modulation de largeur d'impulsion

Firmware Programme embarqué dans le microcontrôleur

OTA Over-The-Air, mise à jour sans fil

9.3 Annexe C : Contacts support

- **Support technique** : Pour tout problème suivant, veuillez contactez l'email correspondant :
 - Script Arduino - tchamijerry@gmail.com
 - Montage - panguipeguy@gmail.com
 - Open Eyes Emergency - aurorengadjou@gmail.com

- Open Eyes Navigation - farellengapgou@gmail.com
- **GitHub du projet** : <https://github.com/JerMa25/OPEN-EYES>

10 Conclusion

Ce guide de déploiement couvre l'ensemble des aspects techniques nécessaires à la mise en œuvre de la solution OPEN EYES. En suivant les étapes décrites, les ingénieurs et techniciens seront en mesure de :

- Installer et configurer le firmware embarqué sur la canne intelligente
- Compiler, générer et déployer les applications mobiles (Navigation App et Emergency App)
- Effectuer les tests de validation du système complet
- Assurer la maintenance et les mises à jour futures

La solution OPEN EYES est conçue pour être robuste, autonome et évolutive. Les futurs développements pourront s'appuyer sur cette base technique solide pour ajouter de nouvelles fonctionnalités et améliorer l'expérience utilisateur.

*Document rédigé pour le projet OPEN EYES
Version 1.0 - 2 mars 2026*